

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Pugazhansan o/c	Reg. No.:	192511341
Section / Batch:	B	Date:	28/08/2026

Code of Conduct

I Pugazhansan o/c (192511341) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Pu

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

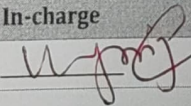
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 - A72

Comparative Analysis1.7) Linear probing vs Separate chaining

Factor	Linear probing	Separate chaining
Clustering	Suffers from primary clustering because consecutive occupied slots form clusters.	No primary clusters, collisions are stored in linked list / buckets.
Memory usage	Lower memory overhead because elements remain inside the hash table.	Higher memory usage due to pointers / nodes for chains.
Search	Average: $O(1)$, Worst: $O(n)$	Average: $O(1)$ with good hashing. Worst: $O(n)$
Insertion	Average: $O(1)$, Worst: $O(n)$	Average: $O(1)$, Worst: $O(n)$
Deletion	More complicated.	Easier.

2) Quick sort VS Merge sort

Factor	Quick sort	Merge sort
Average time	$O(n \log n)$	$O(n \log n)$
Worst time	$O(n^2)$	$O(n \log n)$
Stability	Usually not stable	Stable
Linked list suitability	Less suitable	Highly suitable
Behaviour	Excellent for arrays, but less for the linked list.	Sequential traversal works well with linked lists.

3) Quadratic probing VS Double hashing

Factor	Quadratic probing	Double hashing
Clustering	Avoids primary clustering but can suffer from secondary clustering.	Minimizes both primary and secondary clustering.
Collision resolution	Uses a quadratic sequence such as $h(k) + 1^2, h(k) + 2^2 \dots$	Uses a second hash function to determine the step size.
Efficiency	Better than linear probing but may have clustering.	Generally more efficient and distributes keys better.
Dense tables	Performance decreases as load factor becomes high	Better choice for relatively dense tables, but still requires controlled load factor

Best vs Worst case time complexity of sorting:

Aspect	Best case	Worst case
Meaning	Minimum time required for favourable input.	Maximum time required for unfavourable input.
Practical Significance	Shows how efficient an algorithm can be.	Important for performance guarantees.
Algorithm selection	Useful when inputs are often favourable.	Important when predictable performance is required.
Ex) - Quick sort	$O(n \log n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$

5) Recursive vs Iterative quick sort:

Factor	Recursive quick sort	Iterative quick sort
Stack usage	Uses function call stack, average $O(\log n)$, worst $O(n)$.	Uses an explicit stack, can be controlled to reduce stack usage.
Implementation	Simple and easy to understand.	More complex
Stack overflow risk	Higher, especially with poor pivot selection.	Lower if explicit stack size is managed properly.
Readability	More readable.	Less initiative
Performance	Generally similar.	Generally similar.

Ujjwal